

ESCOLA SUPERIOR DOM HÉLDER CÂMARA

CIÊNCIA DA COMPUTAÇÃO

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES I

CAPÍTULO 1

HISTÓRICO, ABSTRAÇÕES E TECNOLOGIAS COMPUTACIONAIS – Parte II

Arquitetura

- O que é Arquitetura ?
- "É um conjunto de tipos de dados, operações e características de cada nível" - Andrew S. Tanenbaum.
- "Arquitetura de computador refere-se aos atributos de um sistema visíveis a um programador ou, em outras palavras, aqueles atributos que possuem um impacto direto sobre a execução lógica de um programa." - Stallings
- Exemplo: Número de bits para representar dados (bool, int, float), técnicas de endereçamento (físico ou virtual), conjunto de instruções.

Organização

- O que é Organização ?
- “Refere-se às unidades operacionais e suas interconexões que realizam as especificações arquiteturais e como os recursos são implementados” - Stallings
- Exemplo: Tecnologia da memória (RAM, ROM)
- Quando se pensa em projetar um computador ou partes de um, está se pensando em Arquitetura de Computadores.
- A Arquitetura de Computadores utilizada até hoje é a de Von Neumann.

Arquitetura de Von Neumann

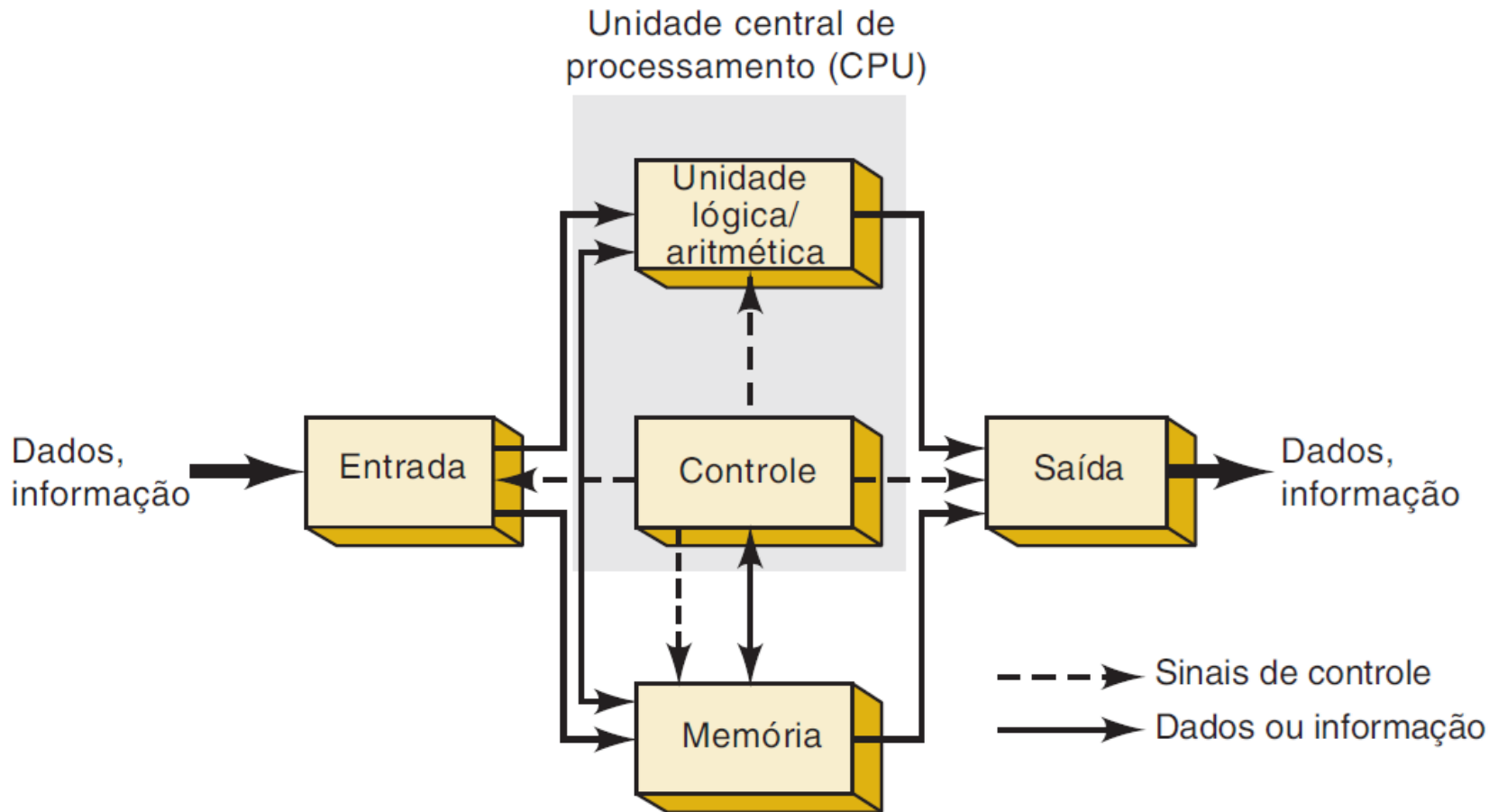
- Se caracteriza pela possibilidade de uma máquina digital armazenar seus programas no mesmo espaço de memória que os dados, podendo assim manipular tais programas.
- Esta arquitetura é um projeto modelo de um computador digital de programa armazenado que utiliza uma unidade de processamento (CPU) e uma de armazenamento ("memória") para comportar, respectivamente, instruções e dados.

Arquitetura de Von Neumann

- A máquina proposta por Von Neumann reúne os seguintes componentes:

1. Uma memória
2. Uma unidade aritmética e lógica (ALU)
3. Uma unidade central de processamento (CPU), composta por diversos registradores, e
4. Uma Unidade de Controle (CU), cuja função é a mesma da tabela de controle da Máquina de Turing universal: buscar um programa na memória, instrução por instrução, e executá-lo sobre os dados de entrada.

Arquitetura de Von Neumann



Arquitetura 80x86

- Visando agilidade na produção para inserção rápida no mercado, pela primeira vez a IBM "terceirizou" a produção dos componentes do seu novo computador para outras companhias.
- O processador escolhido foi o 8086 desenvolvido pela Intel e o sistema operacional foi o DOS (Disk Operating System), desenvolvido por uma companhia de 32 pessoas chamada Microsoft.

Arquitetura 80x86

- Arquitetura de 16 bits
- Comunicação com a memória em 16 bits (8086)
- Capacidade máxima de memória de 1 Mbyte
- Registradores: 14 regs. 4 regs. dados, 4 regs. endereço, 4 regs. segmento, reg. ponteiro do programa, reg. Flags
- Endereço físico = segmento * 16 + deslocamento (!)
- Instruções básicas: 85
- Co-processador: 8087(67 instruções básicas)
- Sem cache, sem memória virtual

Arquitetura 80x86

- Graças a escolha da IBM pelo Intel 8086 para compor o IBM PC, e ao sucesso deste computador, o 8086 passou a ser mundialmente aceito. Muitas companhias (Compaq, Columbia Data Products,...) passaram a copiar a arquitetura do IBM PC e oferecer seus próprios sistemas baseados no 8086.
- Aí começou o mercado de PCs.
- Com este sucesso a Intel passou a focar a maioria de seus investimentos em microprocessadores para PCs e não mais em chips de memória.

Arquitetura 80x86

- 8080: primeiro microprocessador de propósito geral
 - via de dados (barramento interno) de 8 bits
 - Usado no primeiro computador pessoal – Altair
- 8086
 - bem mais poderoso
 - 16 bits
 - cache de instruções, pré-busca de algumas instruções
- 8088: (barramento externo de 8 bits) usado no primeiro IBM PC
- 80286
 - 16 Mbytes de memória endereçável
- 80386
 - 32 bits
 - Suporte para multi-tarefa

Arquitetura 80x86

- 80486
 - esquema de caches sofisticado
 - pipeline de instruções
 - co-processador matemático embutido no chip
- Pentium
 - super-escalar
 - múltiplas instruções executadas em paralelo
- Pentium Pro
 - organização super-escalar aprimorada
 - técnica agressiva de renomeação de registradores (...)
 - previsão de desvios, análise de fluxo de dados, execução
 - especulativa

Arquitetura 80x86

- Pentium II
 - tecnologia MMX (MultiMedia eXtensions)
 - processamento gráfico e de áudio & vídeo
- Pentium III
 - instruções de ponto flutuante adicionais para gráficos 3D
- Pentium 4
 - mais melhorias de ponto flutuante e multimídia
 - mais paralelismo (2005: “dual core” - dois procs. em um)
- Itanium
 - 64 bits (IA-64)
- A partir daí Arquiteturas multicore, Hyperthreading, Superescalar e por aí vai...

Introdução

- Existe uma lacuna entre o que é conveniente para as pessoas e o que é conveniente para computadores:
 - pessoas querem fazer X
 - computadores só podem fazer Y
- Isso gera um grande problema!
- Um dos objetivos deste curso é explicar como esse problema pode ser resolvido.

Introdução

- Duas abordagens e ambas envolvem projetar um conjunto de instruções que é mais conveniente para as pessoas usarem do que o conjunto embutido de instruções de máquina.
- Juntas, essas novas instruções também formam uma linguagem (L1), assim como as instruções de máquina embutidas formam uma linguagem, que chamaremos de L0.
- As duas técnicas diferem no modo como os programas escritos em L1 são executados pelo computador que, afinal, só pode executar programas escritos em sua linguagem de máquina, L0.
- Um método de execução de um programa escrito em L1 é primeiro substituir cada instrução nele por uma sequência equivalente de instruções em L0.
- O programa resultante consiste totalmente em instruções L0.

Introdução

- O computador executa o novo programa L0 em vez do antigo programa L1.
- Essa técnica é chamada de tradução.
- A outra técnica é escrever um programa em L0 que considere os programas em L1 como dados de entrada e os execute, examinando cada instrução por sua vez, executando diretamente a sequência equivalente de instruções L0.
- Essa técnica não requer que se gere um novo programa em L0.
- Ela é chamada de interpretação, e o programa que a executa é chamado de interpretador.
- Tradução e interpretação são semelhantes.
- Nos dois métodos, o computador executa instruções em L1 executando sequências de instruções equivalentes em L0.

Introdução

- Na tradução, o programa L1 inteiro primeiro é convertido para um L0, o programa L1 é desconsiderado e depois o novo L0 é carregado na memória do computador e executado.
- Durante a execução, o programa L0 recém gerado está sendo executado e está no controle do computador.
- Na interpretação, depois que cada instrução L1 é examinada e decodificada, ela é executada de imediato. Nenhum programa traduzido é gerado.
- Aqui, o interpretador está no controle do computador. Para ele, o programa L1 é apenas de dados.
- Ambos os métodos (e cada vez mais, uma combinação dos dois) são bastante utilizados.

Introdução

- Em vez de pensar em termos de tradução ou interpretação, muitas vezes é mais simples imaginar a existência de um computador hipotético ou máquina virtual cuja linguagem seja L1.
- Vamos chamar essa máquina virtual de M1 (e de M0 aquela correspondente a L0).
- Se essa máquina pudesse ser construída de forma barata o suficiente, não seria preciso de forma alguma ter a linguagem L0 ou uma máquina que executou os programas em L0.
- As pessoas poderiam simplesmente escrever seus programas em L1 e fazer com que o computador os executasse diretamente.
- Mesmo que a máquina virtual cuja linguagem é L1 seja muito cara ou complicada de construir com circuitos eletrônicos, as pessoas ainda podem escrever programas para ela.

Introdução

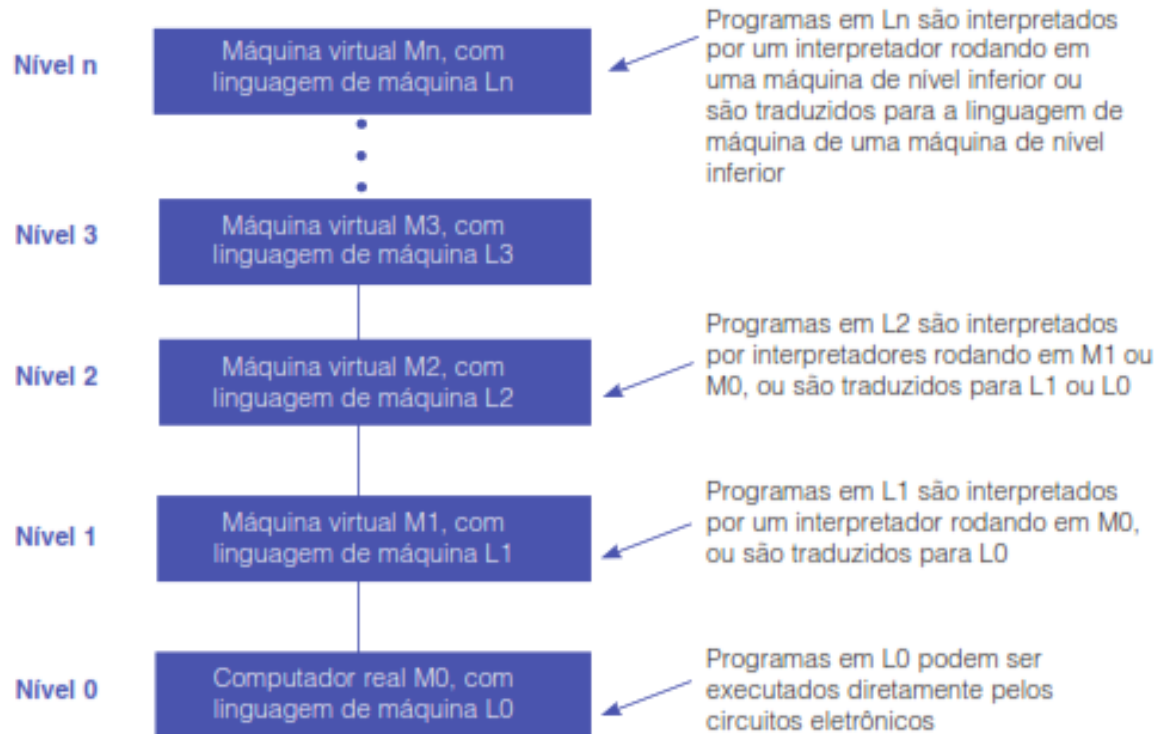
- Esses programas podem ser ou interpretados ou traduzidos por um programa escrito em LM que, por si só, consegue ser executado diretamente pelo computador real.
- Em outras palavras, as pessoas podem escrever programas para máquinas virtuais, como se realmente existissem.
- Para tornar prática a tradução ou a interpretação, as linguagens L0 e L1 não deverão ser “muito” diferentes.
- Tal restrição significa quase sempre que L1, embora melhor que L0, ainda estará longe do ideal para a maioria das aplicações.
- Esse resultado talvez seja desanimador à luz do propósito original da criação de L1 – aliviar o trabalho do programador de ter que expressar algoritmos em uma linguagem mais adequada a máquinas do que a pessoas.

Introdução

- Abordagem óbvia: inventar um conjunto de instruções que seja mais orientado a pessoas e menos orientado a máquinas que a L1.
- Esse terceiro conjunto também forma uma linguagem: L2 (com a máquina virtual M2).
- As pessoas podem escrever programas em L2 exatamente como se de fato existisse uma máquina real com linguagem de máquina L2.
- Esses programas podem ser traduzidos para L1 ou executados por um interpretador escrito em L1.
- A invenção de toda uma série de linguagens, cada uma mais conveniente que suas antecessoras, pode prosseguir indefinidamente, até que, por fim, se chegue a uma adequada.
- Cada linguagem usa sua antecessora como base, portanto, podemos considerar um computador que use essa técnica como uma série de camadas ou níveis.

Introdução

- Máquina Multinível



Introdução

- De certa forma, um computador com n níveis pode ser visto como n diferentes máquinas virtuais, cada uma com uma linguagem de máquina diferente.
- Usaremos os termos “nível” e “máquina virtual” para indicar a mesma coisa.
- Apenas programas escritos na linguagem L_0 podem ser executados diretamente pelos circuitos eletrônicos, sem a necessidade de uma tradução ou interpretação intervenientes.
- Os programas escritos em $L_1, L_2, \dots, L_n$ devem ser interpretados por um interpretador rodando em um nível mais baixo ou traduzidos para outra linguagem correspondente a um nível mais baixo.

Introdução

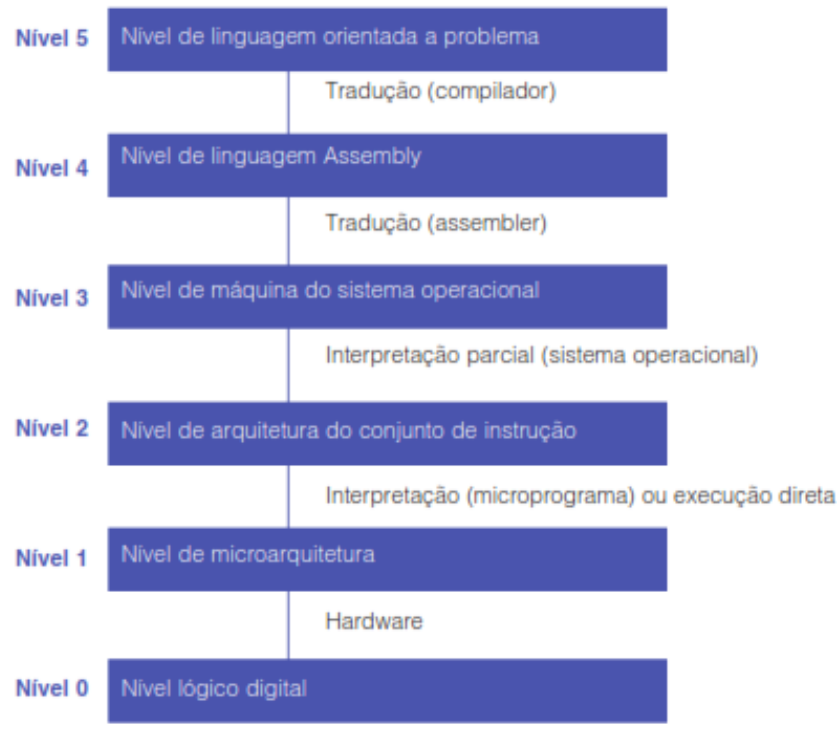
- Uma pessoa que escreve programas para a máquina virtual de nível n não precisa conhecer os interpretadores e tradutores subjacentes.
- A estrutura de máquina garante que esses programas, de alguma forma, serão executados.
- Não há interesse real em saber se eles são executados passo a passo por um interpretador que, por sua vez, também é executado por outro interpretador, ou se o são diretamente pelos circuitos eletrônicos.
- O mesmo resultado aparece nos dois casos: os programas são executados.
- Quase todos os programadores que usam uma máquina de nível n estão interessados apenas no nível superior, aquele que menos se parece com a linguagem de máquina do nível mais inferior.

Introdução

- Porém, as pessoas interessadas em entender como um computador realmente funciona deverão estudar todos os níveis.
- Quem projeta novos computadores ou novos níveis também deve estar familiarizado com outros níveis além do mais alto.

Introdução

- Computador com seis níveis. O método de suporte para cada nível é indicado abaixo dele (junto com o nome que o suporta)



Introdução

- Por que estudar arquitetura de computadores?
 - sempre foi importante projetar sistemas de computação para alto desempenho;
- Características de desempenho básicas dos sistemas computacionais:
 - velocidade de processador
 - velocidade de memória
 - capacidade de memória e
 - taxas de dados de interconexão
- Dificuldade de projetar um sistema balanceado, que maximize o desempenho e a utilização de todos os elementos.
- Técnica: mudar a estrutura ou a função em uma área para compensar uma divergência de desempenho em outra.

Classes de Computadores

- Desktop
 - Enfatizam o bom desempenho a um único usuário por um baixo custo e normalmente são usados para executar software independente.
- Servidor
 - Usado para executar grandes programas para múltiplos usuários, quase sempre de maneira simultânea e normalmente acessado apenas por meio de uma rede.
- Supercomputador
 - Classe de computadores com desempenho e custo mais altos; eles são configurados como servidores.
 - ✓ Clusters de Computadores: warehouse
 - ✓ Paralelismo

Aplicações

- IoT:
 - Computadores embarcados (dentro de outros) que estão conectados à Internet, geralmente sem fio. Quando aumentados com sensores e atuadores, os dispositivos IoT coletam dados úteis e interagem com o mundo físico, levando a uma grande variedade de aplicações “inteligentes”.
- Dispositivo pessoal móvel (PMD)
 - Dispositivos sem fio com interfaces de usuário multimídia
 - Custo é a principal preocupação
 - Eficiência energética
 - Aplicativos para PMDs muitas vezes são baseados na web e orientados para a mídia
 - Processadores em um PMD considerados computadores embarcados

Aplicações

- São plataformas que podem executar software desenvolvido externamente e compartilham muitas das características dos desktop.
- Capacidade de executar softwares de terceiros como a linha divisória entre computadores não embarcados e embarcados.
- A capacidade de resposta e previsibilidade são características-chave para aplicações de mídia. Um requisito de desempenho em tempo real significa que um segmento da aplicação tem um tempo absoluto máximo de execução.

Abaixo do seu programa...

- Os computadores são escravos de nossos comandos $\Rightarrow$ INSTRUÇÃO.
- Exemplo: 1000110010100000 $\Rightarrow$ adição.
- Tanto as instruções quanto os números são representados por conjuntos de dígitos binários.
- Início:
 $\Rightarrow$ programas eram escritos em números binários
 $\Rightarrow$ baixa produtividade e grande quantidade de erros.
- 1ª evolução $\Rightarrow$ linguagem de montagem $\Rightarrow$ uso da máquina para programar a própria máquina.
- Ex.: add A, B $\Rightarrow$ montador $\Rightarrow$ 1000110010100000

Abaixo do seu programa...

- Problema da linguagem de montagem: distante da notação com a qual o ser humano está acostumado a raciocinar.
- 2ª evolução: linguagem de alto nível
- Ex.: $A + B \Rightarrow$ compilador $\Rightarrow$ add A, B $\Rightarrow$ montador 1000110010100000
- Vantagens da linguagem de alto nível:
 - Permitem raciocínio mais próximo do natural do ser humano;
 - São projetadas de acordo com o programa a ser desenvolvido;
 - Aumento da produtividade do programador;
 - Programas tornam-se independentes do hardware;
 - Sanidade mental dos programadores.

Abaixo do seu programa...

- O que é interessante ressaltar:
- O uso da máquina para programar a própria máquina chegou a um nível de abstração tão grande que as máquinas atuais já possuem autonomia até para definir procedimentos corretos ou não.
 - A máquina controla o funcionamento da própria máquina: SO;
 - O sistema operacional é um programa que não tem “nada” de diferente do seu programa?
- Futuro:
 - Máquinas capazes de se autoprogramar.
 - Máquinas capazes de se reproduzir.

O que é um computador?

- Componentes de um computador:
 - Dispositivos de entrada (mouse, teclado)
 - Dispositivos de saída (monitor, impressora)
 - Memória (discos, DRAM, SRAM, CD)
 - Caminho de DADOS.
 - Unidade de CONTROLE.
- Nosso primeiro enfoque: O processador (datapath e controle)

O que é um computador?

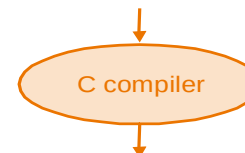
- Abstração: o processador é um VLSI (implementado usando milhões de transistores).
- Impossível de entender olhando cada componente isoladamente.
- Blocos funcionais (caixas)
 - Manter os níveis de detalhes mais baixos escondidos
 - Ex.: a memória é construída com flip-flops, que são construídos com portas lógicas, que são construídas com transistores.

Abstração

- Maior aprofundamento revela mais informações.
- Uma abstração omite detalhes desnecessários, ajudando a lidar com a complexidade do sistema.

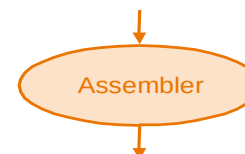
High-level
language
program
(in C)

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```



Assembly
language
program
(for MIPS)

```
swap:
  muli $2, $5, 4
  add $2, $4, $2
  lw $15, 0($2)
  lw $16, 4($2)
  sw $16, 0($2)
  sw $15, 4($2)
  jr $31
```

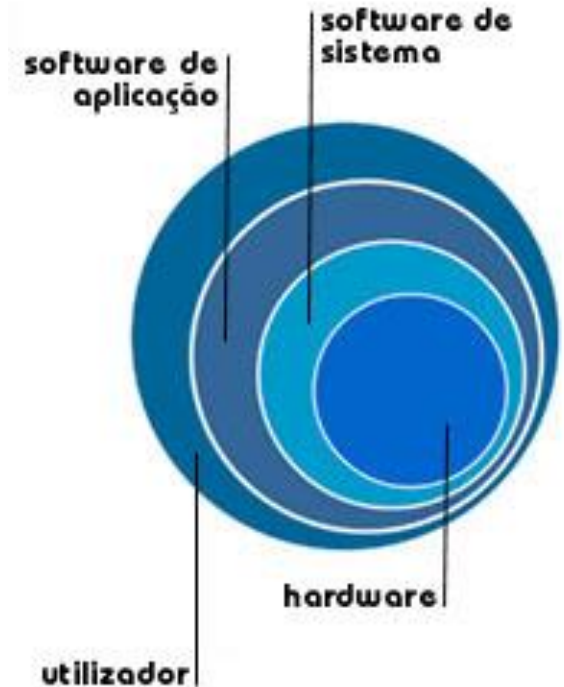


Binary machine
language
program
(for MIPS)

```
000000001010000100000000000011000
00000000100011100001100000100001
10001100011000100000000000000000
100011001111001000000000000000100
101011001111001000000000000000000
101011000110001000000000000000100
000000111110000000000000000001000
```

Por trás do Programa

- Software de sistemas: é o responsável pela gestão de todos os componentes do computador. Fornece serviços que normalmente são úteis, incluindo SsOs, compiladores e montadores.
- Software de aplicação: em regra é criado para executar tarefas específicas (processamento de texto, reprodução de áudio, etc).
- Ao contrário do software de sistema, estas tarefas não são indispensáveis ao normal funcionamento do computador (são executadas a pedido do utilizador).



Por trás do Programa

- O SO fornece a interface entre programa de usuário / hardware e disponibiliza diversos serviços e funções de supervisão.
- Entre as funções mais importantes estão:
 - Manipular as operações básicas de entrada e saída;
 - Alocar armazenamento e memória;
 - Possibilitar e controlar o compartilhamento do computador entre as diversas aplicações que o utilizam simultaneamente.

Mais abstração....

- Uma das abstrações mais importantes é a ARQUITETURA DO CONJUNTO DE INSTRUÇÕES (ISA) ou simplesmente CONJUNTO DE INSTRUÇÕES DE UMA MÁQUINA.
 - É a interface entre hardware e software de baixo nível.
 - É o conjunto de instruções que podem ser utilizadas em determinada máquina.
- Vantagem: admite diferentes implementações da mesma arquitetura.
- Desvantagem: algumas vezes impossibilita o uso de inovações.
- Essa interface abstrata permite que muitas implementações com custo e desempenho variáveis executem um software idêntico.

Arquitetura do Conjunto

- A evolução das máquinas está baseada em dois campos:
 - Melhorias com relação aos materiais de fabricação
 - desempenho do hardware → microeletrônica.
 - Melhorias na organização do funcionamento do computador:
 - Modos de execução das instruções.
 - Paralelismo.
 - Divisão da memória em níveis hierárquicos.
 - Utilização de memória cache.

Definições de Memórias

- Memória volátil: conserva os dados apenas enquanto estiver energizada (DRAM)
- Memória não volátil: conserva os dados mesmo na ausência de energia e que é usada para armazenar programas entre execuções (HD).
- Memória principal: usada para armazenar os programas enquanto estão sendo executados; normalmente consiste na DRAM nos computadores atuais.
- Memória secundária: memória não volátil usada para armazenar programas e dados entre execuções. Normalmente HDs.
- Memória Flash: semicondutora não volátil. Ela é mais barata e mais lenta que a DRAM, porém mais cara e mais rápida que os discos magnéticos.

Definições de Memórias

- Latência: número de ciclos de clock que a memória leva para entregar um dado.

Tecnologia	Velocidade do módulo (MT/s)	Tempo do ciclo de clock (ns)	Latência CAS	Latência (ns)
SDR	100	8,00	3	24,00
SDR	133	7,50	3	22,50
DDR	335	6,00	2,5	15,00
DDR	400	5,00	3	15,00
DDR2	667	3,00	5	15,00
DDR2	800	2,50	6	15,00
DDR3	1333	1,50	9	13,50
DDR3	1600	1,25	11	13,75
DDR4	1866	1,07	13	13,93
DDR4	2133	0,94	15	14,06
DDR4	2400	0,83	17	14,17
DDR4	2666	0,75	18	13,50

<https://br.crucial.com/articles/about-memory/difference-between-speed-and-latency>

Definições de Memórias

- A ambiguidade 2^x versus 10^y bytes foi resolvida acrescentando-se uma notação binária para todos os termos de tamanho comuns.
- Na última coluna, observamos como o termo binário é maior do que seu correspondente decimal, o que é visto quando descemos na tabela.
- Esses prefixos funcionam para bits e também como bytes, portanto gigabit (Gb) é 10^9 bits, enquanto gibibits (Gib) é 2^{30} bits.

Termo decimal	Abreviação	Valor	Termo binário	Abreviação	Valor	% maior
kilobyte	KB	10^3	kibibyte	KiB	2^{10}	2%
megabyte	MB	10^6	mebibyte	MiB	2^{20}	5%
gigabyte	GB	10^9	gibibyte	GiB	2^{30}	7%
terabyte	TB	10^{12}	tebibyte	TiB	2^{40}	10%
petabyte	PB	10^{15}	pebibyte	PiB	2^{50}	13%
exabyte	EB	10^{18}	exbibyte	EiB	2^{60}	15%
zettabyte	ZB	10^{21}	zebibyte	ZiB	2^{70}	18%
zottabyte	YB	10^{24}	yobibyte	YiB	2^{80}	21%

Oito Ideias

1. Projete pensando na Lei de Moore

- A única constante para os projetistas de computador é a mudança rápida, que é controlada em grande parte pela Lei de Moore.
- Ela declara que os recursos do circuito integrado dobram a cada 18 a 24 meses.
- A Lei de Moore resultou de uma previsão desse crescimento na capacidade do CI em 1965, por Gordon Moore, um dos fundadores da Intel.
- Como os projetos de computador podem durar anos, os recursos disponíveis por chip podem facilmente dobrar ou quadruplicar entre o início e o final do projeto.

Oito Ideias

2. Use a abstração para simplificar o projeto

- Os arquitetos e os programadores de computador tiveram que inventar técnicas para que se tornassem mais produtivos, pois, de outra forma, o tempo de projeto aumentaria de modo insustentável quando os recursos aumentassem pela Lei de Moore.
- Uma técnica de produtividade importante para o hardware e o software é usar abstrações para representar o projeto em diferentes níveis de representação; os detalhes de nível mais baixo serão ocultados, para oferecer um modelo mais simples nos níveis mais altos.

Oito Ideias

3. Torne o caso comum veloz

- Tornar o caso comum veloz costuma melhorar mais o desempenho do que otimizar o caso raro.
- Ironicamente, o caso comum normalmente é mais simples do que o caso raro e, portanto, geralmente é mais fácil de melhorar.
- Esse conselho do senso comum implica que você saberá qual é o caso comum, o que é possível apenas com experimentação e medição cuidadosas.

Oito Ideias

4. Desempenho pelo paralelismo

- Desde o nascimento da computação, os arquitetos de computador tem oferecido projetos que geram mais desempenho realizando operações em paralelo.

5. Desempenho pelo Pipelining

- Um padrão de paralelismo em particular é tão prevalecente na arquitetura de computação que merece seu próprio nome: pipelining.

6. Desempenho pela predição

- Seguindo o ditado de que pode ser melhor pedir perdão do que pedir permissão, a última grande ideia a é predição.
- Em alguns casos, na média, pode ser mais rápido prever e começar a trabalhar do que esperar até que você saiba ao certo, supondo que o mecanismo para se recuperar de um erro de previsão não seja tão dispendioso e sua predição seja relativamente precisa.

7. Hierarquia de memórias

- Os programadores desejam que a memória seja rápida, grande e barata, pois a velocidade da memória geralmente modela o desempenho, a capacidade limita o tamanho dos problemas que podem ser resolvidos e o custo da memória, hoje, geralmente é o maior custo do computador.
- Os arquitetos descobriram que podem resolver essas demandas em conflito com uma hierarquia de memórias, com a memória mais rápida, menor e mais cara por bit no topo da hierarquia e a mais lenta, maior e mais barata por bit no fundo.

8. Estabilidade pela redundância

- Os computadores não apenas precisam ser rápidos; eles precisam ser estáveis.
- Como qualquer dispositivo pode falhar, montamos sistemas estáveis incluindo componentes redundantes, que podem assumir o controle quando uma falha ocorre e ajudar a detectá-la.